



WHEATON
COLLEGE

For Christ & His Kingdom

AI for Theological Inquiry

Lecture 3 — Coding agents on an open model, &
vibe coding

Huihuo Zheng
Wheaton College
September 15, 2026

huihuo.zheng@wheaton.edu

Where we are: put the agent loop back around the box

✓ Last week

- Opened the hood on the **raw model**
- One call in, one answer out
- **Caught it inventing a verse**

You met the engine — and its limits.

🤖 This week

- Wrap the **plan** → **act** → **observe** loop back on
- Drive it from a **coding agent** (opencode)
- ... on the **same open ALCF model**

The engine, now in a car that drives itself.

Week 2 = the engine. This week = the engine *plus a loop* — and learning to steer.

Today: wire up a coding agent, then learn to distrust it



Part 1 — Connect

- What a **coding agent** is (the loop, again)
- “It’s just an OpenAI-compatible URL”
- `opcode.json`: one config block
- Launch → pick the ALCF model → run

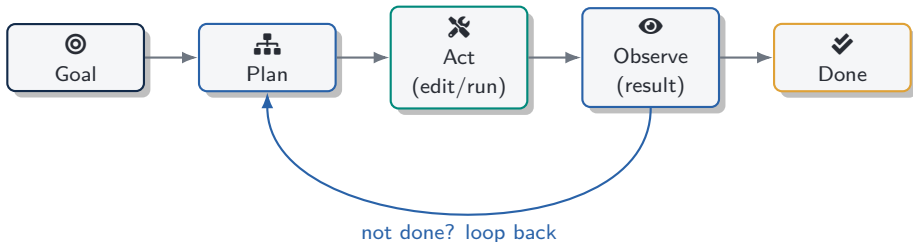


Part 2 — Vibe coding

- What “vibe coding” is — and why it’s fast
- Build a small tool by *describing* it
- Trip the factual trap on purpose
- Vibe to explore, **verify** to publish

~20 min concept · ~25 min connect-lab · ~30 min vibe-lab · ~15 min discipline.

A coding agent is the loop, doing real work



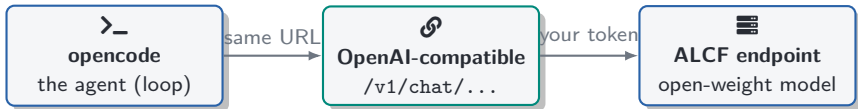
>_ What opencode is

A **free, open-source** coding agent in your terminal. It reads & writes files and runs commands — looping until the goal is met. Same family as Claude Code (Wk 1), but **open** and **provider-agnostic**.

🔑 Why it matters here

We point it at the **same open model on ALCF** from Week 2 — no vendor key, nothing leaves for a company. The stack stays **disclosable**.

The whole trick: the ALCF endpoint is “just a URL”



✓ You already know the hard part

The `base_url` and the token are **the same** as your Week 2 client. A coding agent is just a nicer front-end over that call.

🧩 Three fields to set

`baseUrl` (unchanged) · `apiKey` (your ALCF token, via an env var) · a real `model ID` from `list-endpoints`.

opencode talks to *any* OpenAI-compatible provider — ALCF is one of them.

Step 1 — Give opencode a fresh token (each session)

🔑 Export the token into your shell

```
# same helper as Week 2 (browser login once)
export ALCF_TOKEN="$(python \
    inference_auth_token.py get_access_token)"

# convenience script does exactly this:
source set_alcf_token.sh
```

☰ Confirm a live model ID

```
curl -s https://inference-api.alcf.anl.gov
/\
resource_server/list-endpoints \
-H "Authorization: Bearer $ALCF_TOKEN"
```

Copy an exact ID — you'll paste it into the config.

⚠ opencode reads the token **once, at startup**. When calls start returning **401**, refresh the env var *and restart* opencode. That's not a bug — the token just expired (~48 h).

Step 2 — Tell opencode about ALCF (opencode.json)

opencode.json — one provider, a menu of models

```
{
  "model": "alcf/meta-llama/Meta-Llama-3.1-8B-Instruct", // default at startup
  "provider": { "alcf": {
    "npm": "@ai-sdk/openai-compatible",
    "options": {
      "baseUrl": "https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
      "apiKey": "{env:ALCF_TOKEN}"
    }
  },
  "models": { // every ID here appears in /models
    "meta-llama/Meta-Llama-3.1-8B-Instruct": {},
    "meta-llama/Llama-3.3-70B-Instruct": {},
    "openai/gpt-oss-120b": {}
  }
} }
```

Same baseUrl & token as Week 2 · list as many **served** chat models as you like — each becomes a choice.

Step 3 — Launch, pick the model, run your first agent

▶ Start it

```
opencode # in the project folder

# inside opencode:
/models # menu of ALL models you listed
        # re-run any time to switch
```

Sanity check: “*What model are you, and who serves you?*”

🔗 Pick — and switch

`/models` lists every model from your config. Start with **8B** (fast, and usefully *fallible*); switch to **70B** or `gpt-oss-120b` for harder code. Same agent, same loop — one keystroke.

From here on, you *describe*; the agent plans, writes, runs, and self-corrects.

Part 2 — “Vibe coding”: describe it, let the agent build it

“**Vibe coding** (Karpathy, 2025): say what you want in plain language, let the agent write & run the code, and “*give in to the vibes*” — accepting output **without reading every line**.

⚡ Why it's seductive — and real

Prototype in **minutes**. A low barrier: *describe*, don't syntax. Genuinely great for **throwaway exploration** and learning.

⚠ What it quietly costs

You may **ship code you don't understand**. Bugs and **fabrications** hide behind fluent output that *runs green*.

Same engine as Week 2 — *plausible, not true* — now writing code, not prose.

Warm-ups: three prompts, watch the loop work

1. Plot $\sin(x)$

"Plot $\sin(x)$ on $[0, 2\pi]$ with matplotlib, save `sin.png`, then run it."

No matplotlib? It **installs it and re-runs** — that's the loop.

2. Sort & test

"`sort_ints(nums)` ascending, plus pytest for empty, duplicates, negatives. Run them."

Writes code **and** tests, then **fixes its own** failures.

3. Count words

"Top-10 words in `sample.txt`, minus stop-words."

Now **read** which words it treats as stop-words — a silent decision.

✓ All three **run green** — you described, it built. Next: aim the very same habit at a **factual** claim, and watch it break.

Then trip the trap: green code, confidently wrong

👹 Aim it at a fact

Add `philemon_1_6()` returning the exact text of Philemon 1:6 from memory --- no file, no internet. Add a test that it returns a non-empty string, and run it.

⚠️ What happens

The test **passes** (the string is non-empty) — but the verse is very likely **fabricated**, in fluent biblical cadence. Last week's hallucination, now **baked into code that runs green**.

⚖️ **A passing test proves the code runs, not that it's true. Save it** — prompt, the generated verse, the real verse; diff them. Another disclosure-appendix artifact.

The discipline: vibe to explore, verify to publish

Read

Read what the agent wrote. You are the author; it is a collaborator.

Ground

Tie factual claims to a real source — that's **RAG** (Wk 4).

Test

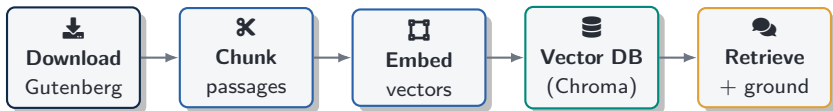
Check that it's right, not just that it runs — **eval** (Wk 6).

Disclose

Report the model, the tool, and what you accepted *unread*.

 **The theological stake:** a fabricated verse in a *script* is worse than in a chat — it looks authoritative, runs automatically, and propagates. Authorship, and responsibility, stay with the scholar.

Capstone: a Luther vector database — RAG on a real corpus



The corpus — public-domain Luther

The **95 Theses**, *Christian Liberty*, the *Large Catechism*, *Table Talk*, *Galatians*... ~8 works, straight from Project Gutenberg.

The integrity test

Ask something **not** in the corpus — it must **refuse**, not invent. Ask for a thesis — the citation must be **verbatim**.

RAG turns the model from a **source** (that fabricates) into a **finder** (that quotes & cites) — you build it by *describing* it (`examples.md`).

Assignment for next week



1. Connect → vibe → catch

Connect **opencode** + **ALCF** and get one agent completion from your open model. Then vibe-code the small tool and **save one fabrication baked into running code** — prompt, the generated (wrong) claim, and the real source.



2. Reading

- Karpathy, *Vibe coding* (2025) — the origin thread; note “forget the code exists.”
- Anthropic, *Building Effective Agents* (2024) — re-skim the tool-use loop.
- Optional: any piece on *reviewing AI-generated code* for research use.

Next week: the theological corpus — the texts *you* will ground on all term.